EigenFaces

355199 Ludovica Ivaldi - 354965 Arianna Giulia Biancone

January 13, 2026

1 Introduction

In this project, we aimed to integrate different topics covered in the course, including Principal Component Analysis (PCA) and Singular Value Decomposition (SVD), and apply them to a real-world problem such as a facial recognition algorithm based on the Eigenfaces approach.
We consider the Olivetti Faces dataset, which consists of grayscale facial images of 40 distinct subjects, with 10 images available for each subject. For the training phase, 9 images per subject are selected, while the remaining image is reserved for testing. The objective of this study is to correctly identify the subject associated with a given test image, while ensuring a computationally efficient recognition procedure.

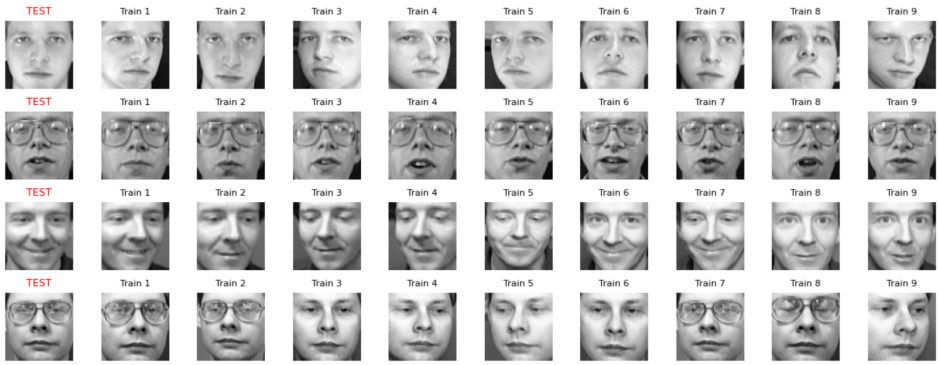


Figure 1: Example of the first four subjects in the dataset, showing both training and test images for each subject.

The methodology followed a progressive approach:

- Initially, a naive solution was implemented in which the entire dataset was stored for recognition.
- To improve computational efficiency, we experimented with representing each subject using the average of their images, thus storing only one image per subject. Although this approach significantly reduced memory usage, it led to a decrease in recognition performance.
- Finally, a change of basis was applied using PCA, selecting the most representative eigenvectors to form a reduced-dimensional subspace. This approach allowed us to achieve a trade-off between recognition accuracy and computational cost.

2 Implemented Methodologies

2.1 Naive Method

A straightforward baseline approach consists in storing all the training images and applying a k-nearest neighbors (KNN) classifier directly in the original image space. For each test image, the Euclidean distance is computed with respect to all 360 training samples, corresponding to 40 subjects with 9

images per subject, and the label of the closest match is assigned.

Each image has a spatial resolution of 64×64 pixels, resulting in a feature vector of $d = 4096$ dimensions. Consequently, the computational cost for classifying a single test image scales linearly with both the number of training samples N and the feature dimensionality d, i.e.,

$$\mathcal{O}(N \cdot d) = \mathcal{O}(360 \cdot 4096).$$

In general, for a dataset containing N training samples of dimensionality d, the classification step requires $\mathcal{O}(N \cdot d)$ operations per test image. This high-dimensional representation leads to significant memory usage and computational cost, rendering the naive approach inefficient and poorly scalable for larger datasets. Although the accuracy is very high (in our work only one test image is misclassified), resulting in an overall accuracy of 0.975.

2.2 Mean-Based Method

The mean-based method reduces computational cost by representing each subject with the pixel-wise average of their 9 training images. This results in a dataset of $N = 40$ representative images, each with $d = 4096$ features. Consequently, the classification of a single test image now requires computing distances to only 40 samples, rather than 360, reducing the computational cost from $\mathcal{O}(360 \cdot 4096)$ to

$$\mathcal{O}(40 \cdot 4096).$$

This efficiency comes at the expense of accuracy: in our experiment, 4 out of 40 test images were misclassified (i.e., accuracy = 0.9), indicating a drop compared to the naive method.

2.3 PCA-Based Method

The PCA-based method leverages Principal Component Analysis (PCA), implemented via Singular Value Decomposition (SVD), to compute the eigenvalues and eigenvectors of the data covariance matrix. By applying a change of basis, the original high-dimensional feature vectors (each corresponding to $d = 4096$ pixel intensities) are transformed into a new set of features known as *eigenfaces*. These eigenfaces capture the principal modes of variation across facial images, providing a more compact and informative representation of the data.

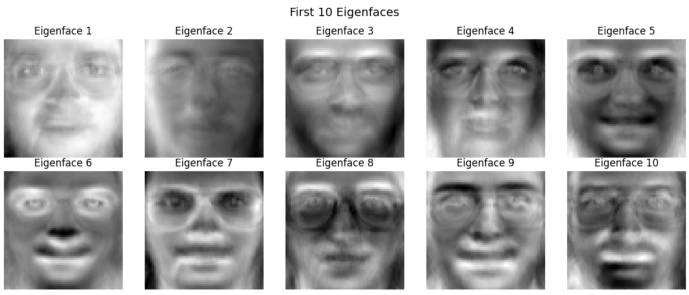


Figure 2: Example of the first ten eigenfaces.

In our experiments, we considered two choices of the number of principal components: $k = 50$ and $k = 7$.

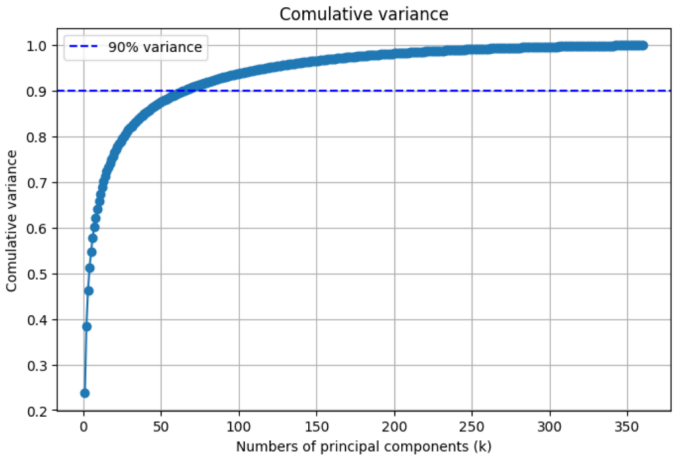


Figure 3: Cumulative variance explained by the first k eigenfaces.

With $k = 50$, the cumulative variance retained was near to 90%. Only 1 out of 40 test images was misclassified, achieving the same accuracy as the naive approach (accuracy = 0.975), while reducing the computational cost from $\mathcal{O}(360 \cdot 4096)$ to

$$\mathcal{O}(360 \cdot 50).$$



Figure 4: The test image misclassified when using $k = 50$ principal components.

By selecting $k = 7$ principal components, we show that the same accuracy as the mean-based approach (0.9) can be achieved using a much lower-dimensional PCA representation. In particular, the PCA-based method results in 4 misclassified test images out of 40.



Figure 5: The four test images misclassified when using $k = 7$ principal components.

From a computational perspective, this leads to a substantial reduction in complexity, as the classification cost decreases from $\mathcal{O}(40 \cdot 4096)$ in the mean-based method to $\mathcal{O}(360 \cdot 7)$ in the PCA-based approach. This result highlights the ability of PCA to preserve recognition performance while significantly reducing computational and memory requirements.

3 Singular Value Decomposition Procedure

We decided not to use the PCA or SVD functions provided by `NumPy`, but instead to implement our own SVD algorithm. The underlying idea is that, by computing the matrices U, Σ, and $V^\top$ of the matrix S, which represents our set of face images, we can directly obtain the eigenvectors of the covariance matrix

$$\mathrm{Cov}(S) = \frac{S^\top S}{N-1}.$$

The corresponding eigenvalues can be obtained from the squares of the singular values of S, divided by $N-1$:

$$\lambda_i = \frac{\sigma_i^2}{N-1}.$$

Since our implementation is designed for matrices A with more rows than columns, we transpose the matrix S. In this setting, the columns of U computed from $A = S^\top$ correspond to the rows of $V^\top$ of the original matrix S.

The fundamental idea behind our implementation is to compute the eigenvectors and eigenvalues of the matrix $A^\top A$ and, from them, to obtain the right and left singular vectors of A as well as the singular values, which correspond to the square roots of the eigenvalues of $A^\top A$.
To facilitate this computation, we first reduce A to a bidiagonal matrix B using the bidiagonalization method we implemented. This is advantageous because if $B = WAT$, then

$$B^\top B = T^\top A^\top W^\top W A T,$$

which is similar to $A^\top A$, but $B^\top B$ has a tridiagonal form. The tridiagonal structure makes the computation of eigenvalues significantly simpler and more efficient.
Once the eigenvectors of $B^\top B$ are computed, we can recover the eigenvectors of $A^\top A$ by a change of basis:

$$Q = T\,[\text{eigenvectors of } B^\top B],$$

where Q represents the matrix of right singular vectors of A with a permutation.

Input: Matrix $A \in R^{m \times n}$, $m \geq n$. **Output:** Matrices U, Σ, V such that $A = U\Sigma V^\top$.

1. Compute the bidiagonal reduction of A:

$$(B, T) \leftarrow \text{bi_diag}(A).$$

2. Compute the eigen-decomposition of the tridiagonal matrix $B^\top B$:

$$(\widetilde{Q}, D) \leftarrow \text{eig_tridiag}(B^\top B).$$

3. Compute the matrix of right singular vectors permuted:

$$Q \leftarrow T\widetilde{Q}.$$

4. Project A onto the right singular subspace:

$$C \leftarrow AQ.$$

5. Compute the QR decomposition with column pivoting:

$$(U, P, R) \leftarrow \text{permuted_qr}(C).$$

6. Set

$$U \leftarrow U, \qquad \Sigma \leftarrow R, \qquad V \leftarrow QP.$$

3.1 Bidiagonalization

The matrix A is reduced to bidiagonal form through a sequence of left and right Householder transformations. There exists two orthogonal matrixs

$$W \in R^{m \times m}$$

$$T \in R^{n \times n}$$

such that

$$B = WAT$$

is bidiagonal, i.e.,

$$B = \begin{pmatrix} \alpha_1 & \beta_1 & & \\ & \alpha_2 & \beta_2 & \\ & & \ddots & \ddots \\ & & & \alpha_n \end{pmatrix}.$$

The only matrix required to proceed with the algorithm is the matrix T, which is obtained as the product of all right Householder transformations applied during the bidiagonalization. To implement this procedure efficiently, we do not explicitly compute the product of all Householder matrices at each iteration, since the effective size of T decreases over the iterations. Instead, we initialize T as an identity matrix of size $n \times n$ and, at each step, perform the updates only on the relevant bottom-right submatrix corresponding to the active portion of the transformation.

3.1.1 Householder Transformation

As previously mentioned, both left and right Householder reflections must be applied to the matrix A in order to reduce it to bidiagonal form B. The left Householder reflection eliminates the elements below the main diagonal, one column at a time, whereas the right Householder reflection eliminates elements in a row starting from the second entry after the diagonal.
A Householder matrix is constructed for each vector that we aim to zero out. Given a vector x, the corresponding Householder vector v is defined as

$$v_0 = \operatorname{sgn}(x_0)\left(|x_0| + \|x\|\right), \qquad v_i = x_i \text{ for } i > 0.$$

The Householder matrix, when v is normalized, is given by

$$H = I - 2vv^\top,$$

which satisfies

$$Hx = (\pm\|x\|, 0, \ldots, 0)^\top.$$

For an efficient implementation, we do not explicitly construct the Householder matrix at each iteration. Instead, we store only the Householder vector and perform the necessary operation directly:

$$WA = A - 2v(v^\top A), \qquad AT = A - 2(Av)v^\top,$$

so that the computational cost is limited to a matrix-vector product rather than a full matrix-matrix product.

Moreover, the procedure can be further optimized. Since the effect of the Householder transformation is known in advance (the first element becomes $\|x\|$ and the remaining entries are zero), we can directly insert these values into the matrix. This also allows us to reduce the effective size of A on which the operation is performed, improving computational efficiency.

3.2 Tridiagonal Eigenvalue Problem

From the previous function, we obtain the matrices B and T. We can now compute the eigenvalues and eigenvectors of $B^\top B$, which is a tridiagonal matrix. The tridiagonal structure allows for a straightforward and efficient computation. Since $B^\top B$ is similar to $A^\top A$, the eigenvalues of $B^\top B$ correspond directly to those of $A^\top A$, and the eigenvectors of $A^\top A$ (i.e., the right singular vectors of A) can be recovered via a change of basis:

$$Q = T\,[\text{eigenvectors of } B^\top B].$$

This computation is implemented using the native `numpy` function `eigh_tridiagonal`, which is specifically designed for tridiagonal matrices and ensures both efficiency and numerical stability.

3.3 QR Decomposition with Column Pivoting

Once we have obtained the matrix of eigenvectors of $A^\top A$, namely Q, we can proceed by computing

$$C = AQ.$$

A QR decomposition with column pivoting is then performed:

$$CP = UR,$$

where the pivoting ensures that the diagonal elements of R are ordered from the largest to the smallest. In this context, R corresponds to the Σ matrix in the SVD decomposition.

We rely on the native implementation of this decomposition. However, it is worth noting that QR decomposition can be performed using Householder reflections. Since the only element that does not become zero after a reflection is the norm of the corresponding vector, the columns can be ordered beforehand according to their norms to achieve the desired ordering.

At the end of this procedure, all the matrices required for the SVD are obtained: the orthogonal matrix $U \in R^{m\times m}$, the diagonal matrix R which represents $\Sigma \in R^{m\times n}$, and the orthogonal matrix $V \in R^{n\times n}$, which can be recovered by permuting Q as $V = QP$.

Although R is obtained as an upper-triangular matrix from the QR decomposition, it can be shown that it is in fact diagonal. Indeed, since $C = AQ$, we have

$$A = CQ^\top = (CP)P^\top Q^\top = UR(QP)^\top.$$

Consequently,

$$A^\top A = QPR^\top U^\top URP^\top Q^\top = QPR^\top RP^\top Q^\top.$$

On the other hand, $A^\top A = QDQ^\top$, where D is diagonal. Therefore,

$$PR^\top RP^\top = D,$$

which implies that $R^\top R$ is diagonal. Since P is merely a permutation matrix, it follows that R itself must be diagonal.

4 Conclusion

In this study, we implemented and compared three facial recognition approaches: the naive method, the mean-based method, and the PCA-based method. Table 1 summarizes their accuracy and computational cost per test image.

Method	Accuracy	Feature Dimensionality	Cost per Test Image
Naive	0.975	$d = 4096$	$\mathcal{O}(360 \cdot 4096)$
Mean-based	0.9	$d = 4096$	$\mathcal{O}(40 \cdot 4096)$
PCA ($k = 50$)	0.975	$k = 50$	$\mathcal{O}(360 \cdot 50)$
PCA ($k = 7$)	0.9	$k = 7$	$\mathcal{O}(360 \cdot 7)$

Table 1: Comparison of recognition methods in terms of accuracy, feature dimensionality, and computational cost per test image.

These results highlight the following points:

- The naive method achieves the highest accuracy but at a high computational cost.
- The mean-based method reduces computational cost but sacrifices accuracy.
- PCA provides a flexible trade-off: with $k = 50$ components, it retains the accuracy of the naive method at much lower cost, and with $k = 7$ components, it achieves the same accuracy as the mean-based method while drastically reducing feature dimensionality and computational burden.

Overall, dimensionality reduction via PCA proves to be an effective strategy for maintaining high recognition performance while substantially decreasing both computational and memory requirements.